

CSC 2400-002: Design of Algorithms (Spring 2026)

Tennessee Tech University

Drill 2 Solutions

Topic: Introduction to Algorithms, Maximum Finding

ONLY FOR THE EYES OF STUDENTS IN CSC 2400 SECTION 002, SPRING 2026, TENNESSEE TECH UNIVERSITY. IF YOU ARE NOT PART OF THIS CLASS, PLEASE STOP READING IMMEDIATELY AND DELETE/DESTROY THIS FILE. A STUDENT SHARING THIS SOLUTION KEY WITH SOMEONE OUTSIDE THE CLASS/COURSE STAFF CONSTITUTES STUDENT ACADEMIC MISCONDUCT.

Question 1. In class, we designed an algorithm to find the largest element in an array. Now write the pseudocode for an algorithm to find the smallest element in an array A of n integers, given by $A[1], A[2], \dots, A[n]$. You must precisely mention the input and the output in the beginning.

Solution:

Algorithm 1 Finding the smallest element in an array

Input: Array A of n integers, given by $(A[1], \dots, A[n])$

Output: The smallest element in A

```
1: curr_min  $\leftarrow A[1]$ 
2: for  $i = 2$  to  $n$  do:
3:     if  $A[i] < \text{curr\_min}$  then:
4:         curr_min  $\leftarrow A[i]$ 
5: return curr_min
```

Question 2. In class, we saw an algorithm that finds the largest element in an array by making $n - 1$ comparisons between array elements. Now describe an algorithm in English (pseudocode not needed) that, given an input array A of n integers, makes at most $(n - 1) + (n - 2)$ comparisons and outputs both the minimum element and the maximum element of A . Give a brief justification for why the number of comparisons made by your algorithm is as desired.

Solution: Given an array $A = (A[1], \dots, A[n])$, we compare the first two elements $A[1]$ and $A[2]$, and set the smaller number as the current minimum and the larger one as the current maximum (if they have equal value, then both the current maximum and minimum have that value). Then, for each $i = 3, \dots, n$, we compare $A[i]$ with both the current minimum and the current maximum. We reset the current minimum (respectively, maximum) to $A[i]$ if $A[i]$ turns out to be smaller (respectively, larger). After scanning all array elements, we return the value stored in the current minimum (resp. maximum) as the minimum (resp. maximum) element of the array. Thus, we make $1 + (n - 2) + (n - 2) =$

$(n - 1) + (n - 2)$ comparisons in total.

Note that we saved on one comparison over the naive $2(n - 1)$ since we need to compare the first two elements only once to initialize both the current maximum and the current minimum. We then iterate over elements $A[2], \dots, A[n - 1]$ as before.